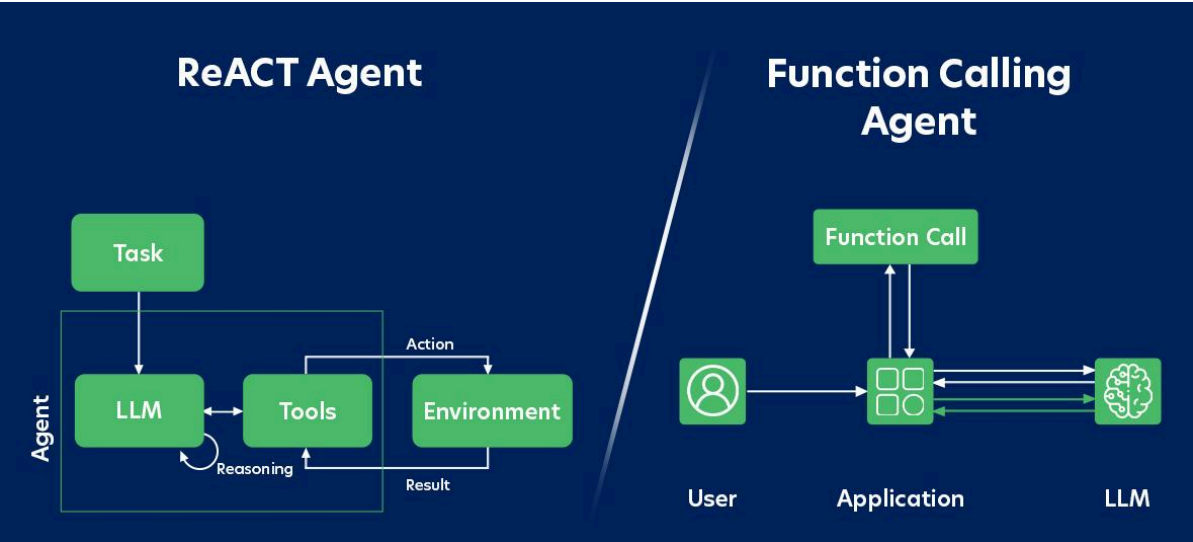


ReAct范式深度解析



引言

1. 做智能解决方案系统时，如何让AI在复杂任务中既保持推理能力，又能有效执行行动？传统AI系统往往要么只能基于训练数据推理，要么只能执行固定流程，缺乏动态决策能力。
2. ReAct (Reasoning and Acting) 范式正是为了解决这个问题而诞生的。它让AI能够交替进行推理和行动，通过"思考-行动-观察-调整"的循环，实现更智能的决策过程。

一、ReAct范式原理

1.1 ReAct概念

ReAct范式由Shunyu Yao等人在2022年的论文《ReAct: Synergizing Reasoning and Acting in Language Models》中首次提出。

ReAct的核心在于推理和行动的交替进行，而不是传统AI的"纯推理"或"纯行动"模式。

通过一个具体的天气查询例子来说明：

```
# 传统AI方法：要么纯推理，要么纯行动
def traditional_reasoning_only(question):
    """纯推理方法：仅基于训练数据回答"""
    return "基于我的训练数据，今天可能是晴天"

def traditional_action_only(question):
    """纯行动方法：直接调用API，缺乏思考"""
    # 模拟直接调用API，没有推理过程
```

```
if "天气" in question:
    return "晴天，温度25°C" # 硬编码结果，没有推理
return "无法处理"

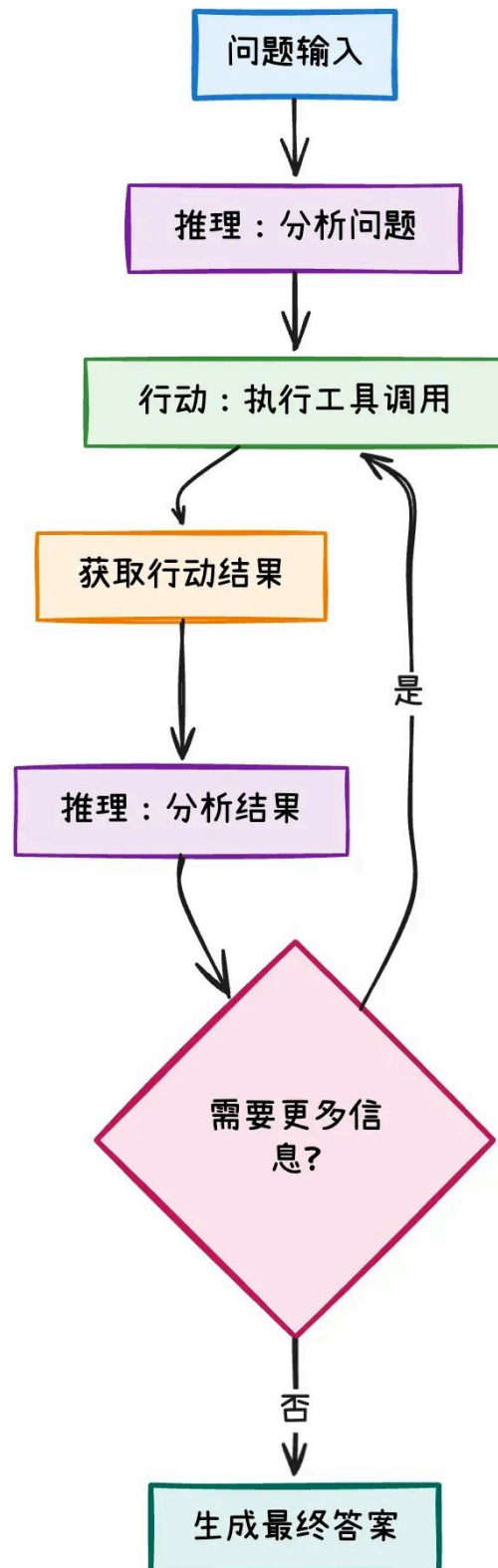
# ReAct方法：推理和行动交替进行
def react_approach(question):
    """ReAct方法：推理和行动交替进行"""
    # 第1步：推理 - 分析问题
    reasoning = "用户问的是今天某城市的天气，我需要查询实时天气信息"

    # 第2步：行动 - 执行查询
    weather_result = weather_api("某城市")

    # 第3步：推理 - 分析查询结果
    reasoning = "查询结果显示今天某城市是晴天，温度25度，这是实时准确信息"

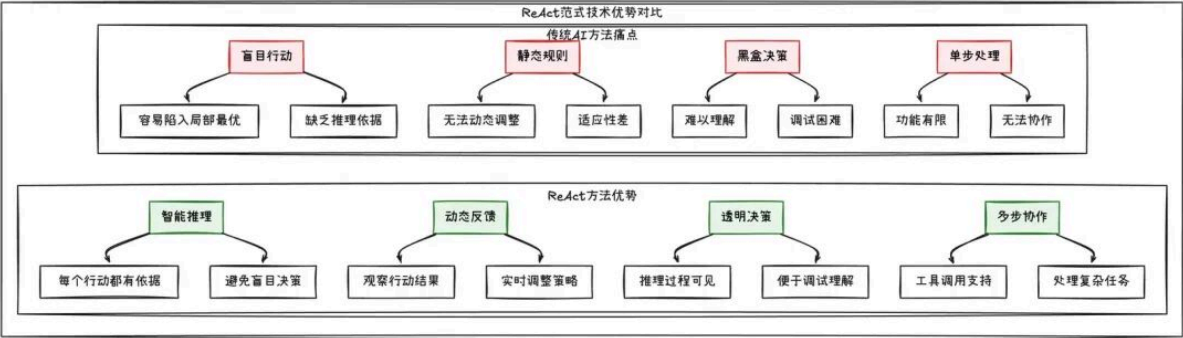
    # 第4步：行动 - 生成最终答案
    return "今天某城市是晴天，温度25度，适合外出"
```

ReAct让AI能够在需要时主动获取信息，而不是仅依赖训练数据，同时保持推理过程的透明性。



1.2 ReAct范式为什么有效

ReAct范式之所以有效，在于它解决了传统AI方法的几个问题：



解决信息获取问题

传统AI只能依赖训练数据，ReAct让AI能够主动获取最新信息，解决知识时效性问题。

实现推理与行动的结合

不是简单的"先推理后行动"，而是推理和行动的交替进行，让AI能够根据中间结果调整策略。

保持推理过程透明

每个行动都有明确的推理依据，推理过程完全可见，便于调试和理解AI的决策逻辑。

支持复杂协作

通过工具调用机制，ReAct可以处理需要多步骤、多工具协作的复杂任务场景。

1.3 ReAct的设计理念

ReAct范式不仅仅是一个技术方案，它反映了AI系统设计理念的转变：

从"黑盒AI"到"透明AI"

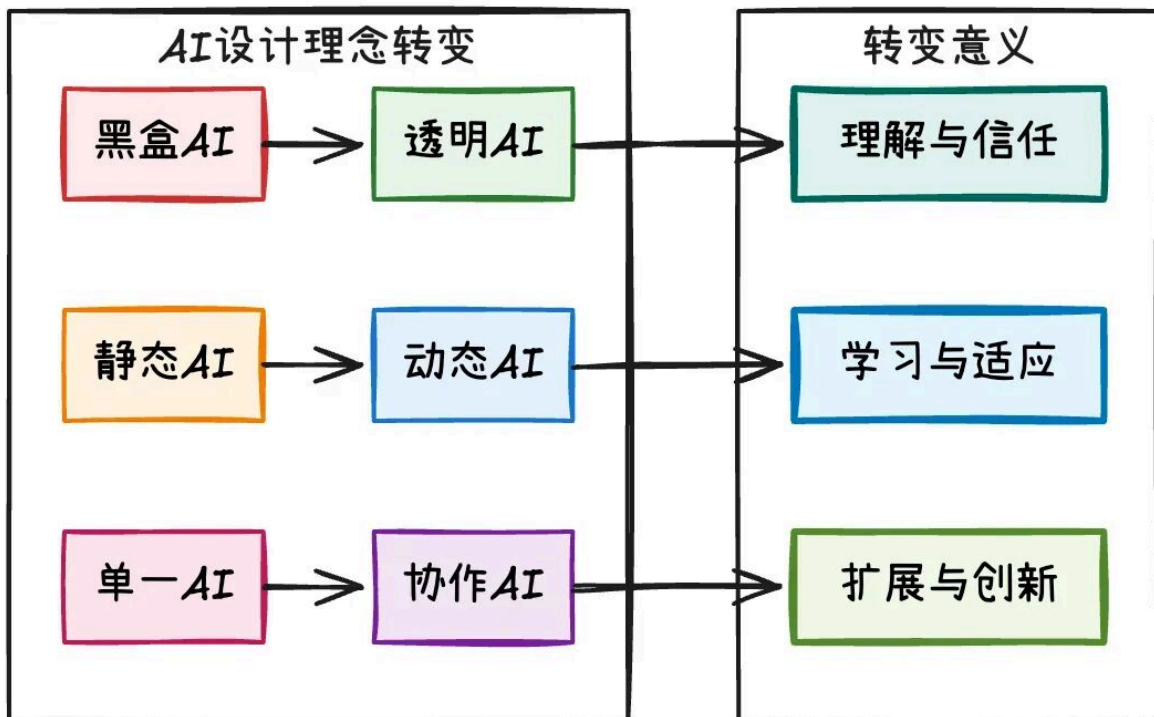
传统AI的决策过程是黑盒的，用户无法理解AI为什么做出某个决定。ReAct让AI的思考过程变得透明，每个行动都有明确的推理依据，这带来了理解与信任。

从"静态AI"到"动态AI"

传统AI只能基于训练时的静态数据，无法适应新情况。ReAct让AI能够主动获取最新信息，根据实际情况动态调整策略，实现了学习与适应。

从"单一AI"到"协作AI"

传统AI往往是孤立的，无法与其他系统协作。ReAct通过工具调用机制，让AI能够与其他系统协作，实现了扩展与创新。

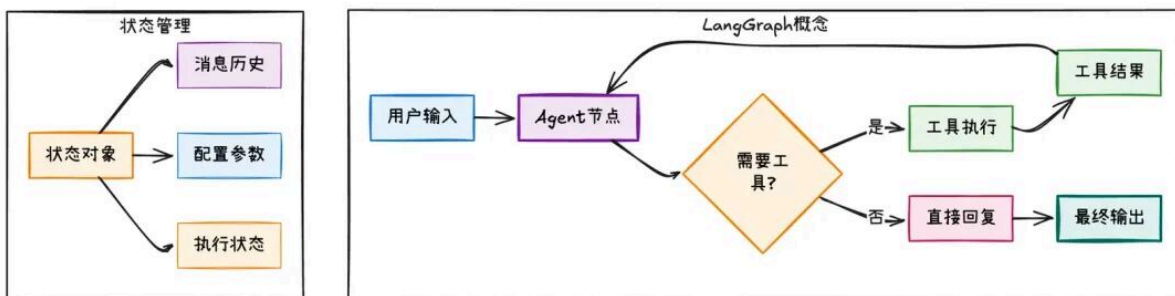


理解了ReAct的原理后，你可能会想：如何在项目中实现这种"推理-行动"的循环？比如，如何让AI在需要时主动调用工具，如何管理整个对话状态，如何控制循环的结束条件？LangGraph就是专门解决这些问题的框架。

二、LangGraph中的ReAct实现机制

2.1 什么是LangGraph?

LangGraph是LangChain团队开发的用于构建AI Agent的框架。它的思想是：将AI Agent的执行过程抽象为一个有向图。



LangGraph的核心特性：

- 图结构：**将AI逻辑抽象为节点和边的图；
- 状态驱动：**系统围绕状态对象运行；
- 条件路由：**根据状态决定下一步执行路径；

2.2 LangGraph如何实现ReAct?

通过一个完整的例子来理解LangGraph如何实现ReAct范式：

```
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage

# 1. 定义工具
@tool
def search_weather(location: str) -> str:
    """搜索指定地点的天气信息"""
    # 模拟天气查询API
    weather_data = {
        "A城市": "晴天, 温度25°C, 湿度60%",
        "B城市": "多云, 温度22°C, 湿度70%"
    }
    return weather_data.get(location, f"{location}的天气信息暂时无法获取")

@tool
def calculate_distance(city1: str, city2: str) -> str:
    """计算两个城市之间的距离"""
    # 模拟距离计算API
    distances = {
        ("A城市", "B城市"): "约500公里",
        ("B城市", "A城市"): "约500公里"
    }
    return distances.get((city1, city2), f"{city1}到{city2}的距离信息暂时无法获取")

# 2. 创建模型
model = ChatOllama(model="qwen3:8b", temperature=0.1)

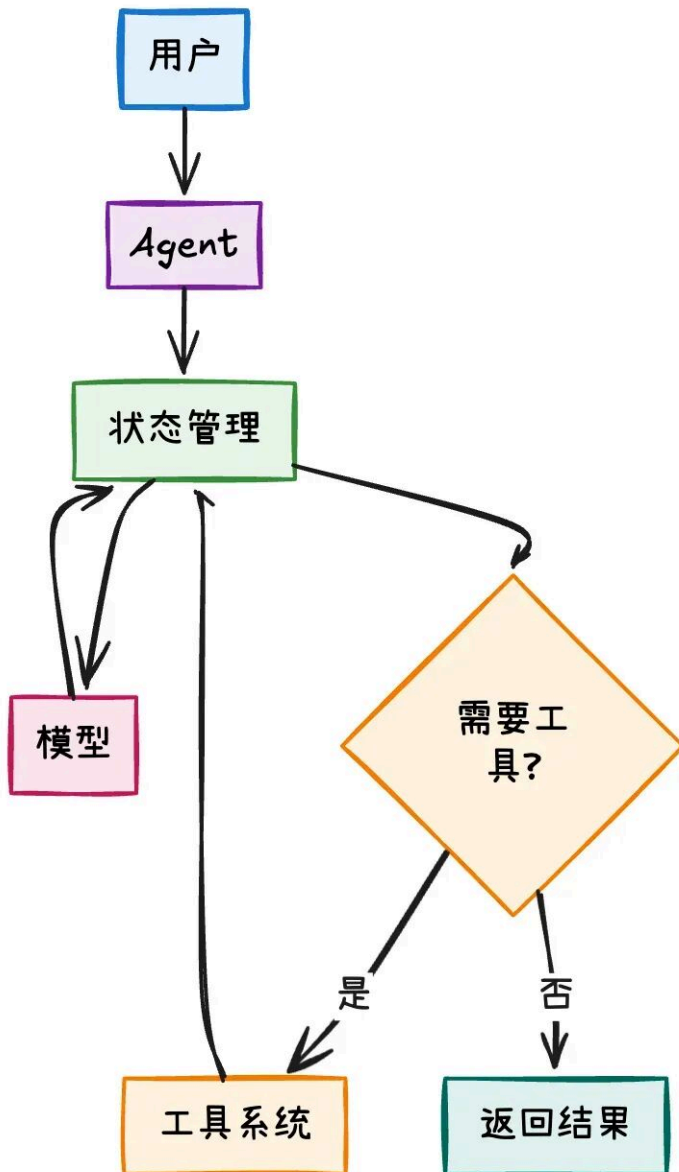
# 3. 创建ReAct Agent
agent = create_react_agent(
    model=model,
    tools=[search_weather, calculate_distance],
    prompt="你是一个天气助手, 可以帮助用户查询天气和计算距离。",
    version="v2")

# 4. 使用Agent
result = agent.invoke({
    "messages": [HumanMessage(content="A城市和B城市的天气怎么样? 距离有多远?")]
})
```

LangGraph的简洁性体现在：几行代码就创建了一个完整的ReAct Agent，系统自动判断是否需要调用工具，消息历史自动维护，工具调用完全透明。

2.3 LangGraph的内部机制

让我们深入看看 `create_react_agent` 内部是如何工作的：



核心实现逻辑：

```
# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
def create_react_agent(
    model: BaseChatModel,
    tools: Sequence[BaseTool],
    prompt: Optional[BaseMessage] = None,
    response_format: Optional[Union[Dict, Type[BaseModel]]] = None,
    pre_model_hook: Optional[Callable] = None,
    post_model_hook: Optional[Callable] = None,
    state_schema: Optional[Type[TypedDict]] = None,
    version: Literal["v1", "v2"] = "v2",
) -> CompiledGraph:
    """创建ReAct Agent的核心实现"""
    # 1. 定义状态结构
    class AgentState(TypedDict):
        messages: Annotated[Sequence[BaseMessage], add_messages]
```

```

        remaining_steps: NotRequired[RemainingSteps]

# 2. 创建图结构
workflow = StateGraph(AgentState)

# 3. 定义节点
def agent_node(state: AgentState):
    """Agent节点: 调用模型进行推理"""
    model_with_tools = model.bind_tools(tools)
    response = model_with_tools.invoke(state["messages"])
    return {"messages": [response]}

def tools_node(state: AgentState):
    """工具节点: 执行工具调用"""
    return tool_node.invoke(state)

# 4. 定义路由逻辑
def should_continue(state: AgentState) -> str:
    """判断是否需要继续执行工具"""
    last_message = state["messages"][-1]
    if hasattr(last_message, 'tool_calls') and last_message.tool_calls:

        return "tools"
    return "end"

# 5. 组装图结构
workflow.add_node("agent", agent_node)
workflow.add_node("tools", tools_node)
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {"tools": "tools", "end": END}
)
workflow.add_edge("tools", "agent")

return workflow.compile()

```

2.4 关键设计思想

1. 状态驱动架构

```

# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
# 状态是系统的核心
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], add_messages] # 消息历史
    remaining_steps: NotRequired[RemainingSteps] # 剩余步数

```

状态包含所有必要信息，支持断点续传，便于调试和监控。

2. 条件路由机制

```
# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
def should_continue(state: AgentState) -> str:
    """智能路由决策"""
    last_message = state["messages"][-1]
    if hasattr(last_message, 'tool_calls') and last_message.tool_calls:
        return "tools" # 需要工具
    return "end" # 直接结束
```

基于内容而非规则，支持复杂决策逻辑，易于扩展和修改。

3. 工具绑定机制

```
# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
# 工具绑定到模型
model_with_tools = model.bind_tools(tools)
response = model_with_tools.invoke(messages)
```

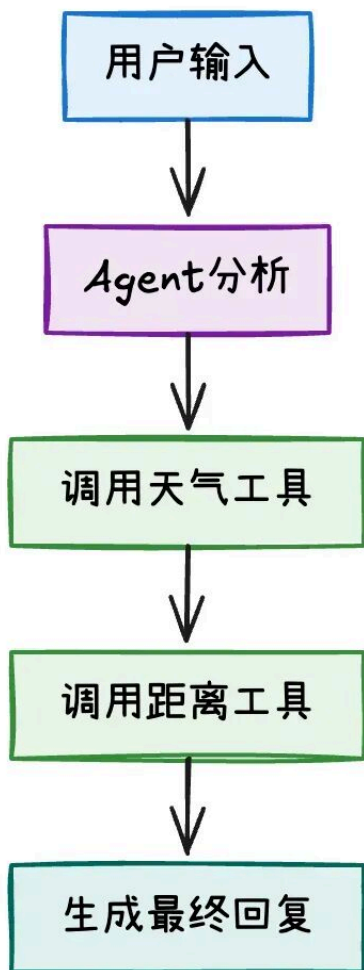
工具调用对AI透明，支持并行工具调用，提供统一的工具接口。

2.5 实际运行示例

让我们看一个完整的运行过程：

```
# 用户输入
user_input = "A城市和B城市的天气怎么样？距离有多远？"
# 执行过程
result = agent.invoke({
    "messages": [{"role": "user", "content": user_input}]
})
# 执行步骤：
# 1. Agent接收用户消息
# 2. 模型分析：需要查询天气和计算距离
# 3. 调用search_weather工具
# 4. 调用calculate_distance工具
# 5. 基于工具结果生成最终回复
```

执行流程图：



2.6 设计特点总结

LangGraph的设计有几个特点：

简单易用： 几行代码创建Agent，自动处理状态管理，路由决策。

灵活： 支持自定义工具，可扩展的图结构，钩子机制。

性能优化： 并行工具调用，状态增量更新，缓存机制。

易于调试： 执行日志，状态可视化，错误追踪。

`create_react_agent` 是LangGraph中的重要函数，它封装了ReAct范式的实现。

三、create_react_agent源码解析

3.1 实现逻辑

```
# 文件路径: libs/prebuilt/langgraph/prebuilt/chat_agent_executor.py
def create_react_agent(
    model: Union[str, LanguageModelLike, Callable],
    tools: Union[Sequence[BaseTool], ToolNode],
    prompt: Optional[Prompt] = None,
    response_format: Optional[StructuredResponseSchema] = None,
    state_schema: Optional[StateSchemaType] = None,
```

```

        version: Literal["v1", "v2"] = "v2",
    ) -> CompiledStateGraph:
        """
        创建ReAct Agent的核心函数

        参数说明:
        - model: 语言模型实例
        - tools: 工具列表或工具节点
        - prompt: 自定义提示词
        - response_format: 结构化输出格式
        - state_schema: 状态模式
        - version: 版本选择
        """

```

关键设计点:

1. 状态模式的动态选择

```

# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
if state_schema is None:
    state_schema = (
        AgentStateWithStructuredResponse
        if response_format is not None
        else AgentState
    )

```

简单场景使用基础状态，复杂场景支持结构化输出，避免了过度设计。

2. 工具绑定的智能判断

```

# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
if _should_bind_tools(model, tool_classes, num_builtin=len(llm_builtin_tools)):

    model = model.bind_tools(tool_classes + llm_builtin_tools)

```

能自动判断模型是否需要绑定工具，处理不同类型的工具，用户不需要关心技术细节。

3. 路由逻辑的单一职责

```

# 文件路径: libs/langgraph/langgraph/prebuilt/agent_executor.py
def should_continue(state: StateSchema) -> Union[str, list[Send]]:
    messages = _get_state_value(state, "messages")
    last_message = messages[-1]
    if not isinstance(last_message, AIMessage) or not last_message.tool_calls:

        return END
    else:
        return "tools"

```

只负责路由决策，基于内容而非规则，支持扩展（钩子、结构化输出）。

3.2 设计原理深度分析

1. 状态管理的设计思路

```
# 传统方法：全局状态，难以管理
class TraditionalAgent:
    def __init__(self):
        self.messages = []
        self.tool_results = []
        self.current_step = 0
        # ... 更多状态变量

# LangGraph方法：状态模式，类型安全
# 文件路径：libs/langgraph/langgraph/prebuilt/agent_executor.py
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], add_messages]
    remaining_steps: NotRequired[RemainingSteps]
```

LangGraph使用TypedDict确保状态类型安全，使用Annotated提供状态更新策略，让状态管理既安全又灵活。

2. 条件路由的智能实现

```
# 文件路径：libs/langgraph/langgraph/prebuilt/agent_executor.py
def should_continue(state: AgentState) -> str:
    messages = state["messages"]
    last_message = messages[-1]

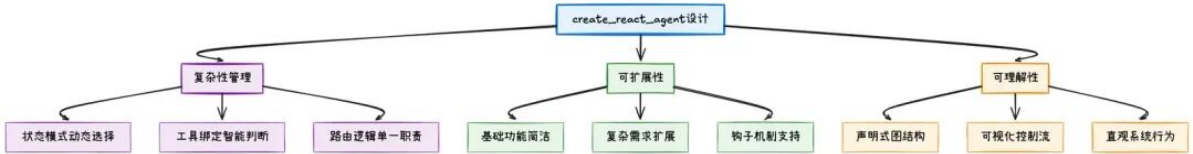
    # 关键设计：不是检查所有消息，而是只检查最后一条
    if isinstance(last_message, AIMessage) and last_message.tool_calls:
        return "tools"
    return "end"
```

只检查最后一条消息，避免了遍历整个消息历史，性能更好，逻辑更清晰。

3. 工具绑定的统一抽象

```
# 文件路径：libs/langgraph/langgraph/prebuilt/agent_executor.py
# 核心设计：工具绑定到模型，而不是单独管理
model_with_tools = model.bind_tools(tools)
response = model_with_tools.invoke(state["messages"])
```

通过工具绑定机制，让AI能够调用工具而不需要了解工具的具体实现细节。



1. 复杂性管理

ReAct系统天然复杂，但通过精妙的设计，让复杂性变得可控。

2. 可扩展性

基础功能保持简洁，复杂需求通过扩展支持。

3. 可理解性

通过声明式的图结构，开发者可以直观地理解系统行为。

说明

了解 LangGraph的设计原理后，让我们通过一个真实的项目案例来看看ReAct在实际应用中的价值。这个案例来自智能解决方案系统，其中PPT生成大纲是其中的一个重点能力。

四、真实项目案例：智能解决方案系统

4.1 项目背景：为什么选择ReAct?

在智能解决方案系统中，我选择使用ReAct范式构建智能大纲生成Agent。这个决策背后有一个真实的踩坑故事：

第一阶段：纯大模型方案的问题

```
# 最初的简单方案
def generate_outline(user_input):
    """纯大模型方案：直接生成PPT大纲"""
    prompt = f"""
    用户需求: {user_input}
    请生成一份完整的PPT大纲，包含：
    1. 封面页
    2. 目录页
    3. 产品介绍
    4. 功能特性
    5. 应用案例
    6. 总结页
    """
    return llm.invoke(prompt)
```

问题暴露：

- 客户信息不准确：大模型不知道具体的客户背景；
- 产品信息过时：内部产品信息无法实时更新；
- 用户需求不明确：用户可能只需要某几页，不需要完整PPT；

第二阶段：增加知识检索的尝试

```
# 改进方案：增加知识检索
def generate_outline_with_knowledge(user_input):
    # 检索客户信息
    customer_info = search_customer_knowledge(user_input)
    # 检索产品信息
    product_info = search_product_knowledge(user_input)
    # 检索行业信息
    industry_info = search_industry_knowledge(user_input)
```

```
prompt = f"""
用户需求: {user_input}
客户信息: {customer_info}
产品信息: {product_info}
行业信息: {industry_info}
请生成一份完整的PPT大纲...
"""

return llm.invoke(prompt)
```

新问题出现：

- 信息过载：每次都要检索大量信息，即使不需要；
- 固化输出：总是生成"完整"的PPT大纲；
- 用户体验差：用户说"我只要3页介绍产品功能"，却生成了20页的完整大纲；

能不能让AI像人类一样，先分析用户真正需要什么，再决定要不要检索信息，检索什么信息，生成什么样的结构？

第三阶段：ReAct范式的解决方案

```
# ReAct方案：智能判断
def react_outline_generation(user_input):
    # 1. 推理：分析用户真正需要什么
    reasoning = analyze_user_intent(user_input)
    # 2. 行动：根据推理结果决定是否检索
    if reasoning.needs_customer_info:
        customer_info = search_customer_knowledge(user_input)
    if reasoning.needs_product_info:
        product_info = search_product_knowledge(user_input)
    # 3. 观察：基于检索结果调整策略
    # 4. 调整：生成符合用户真实需求的大纲
    return generate_adaptive_outline(reasoning, retrieved_info)
```

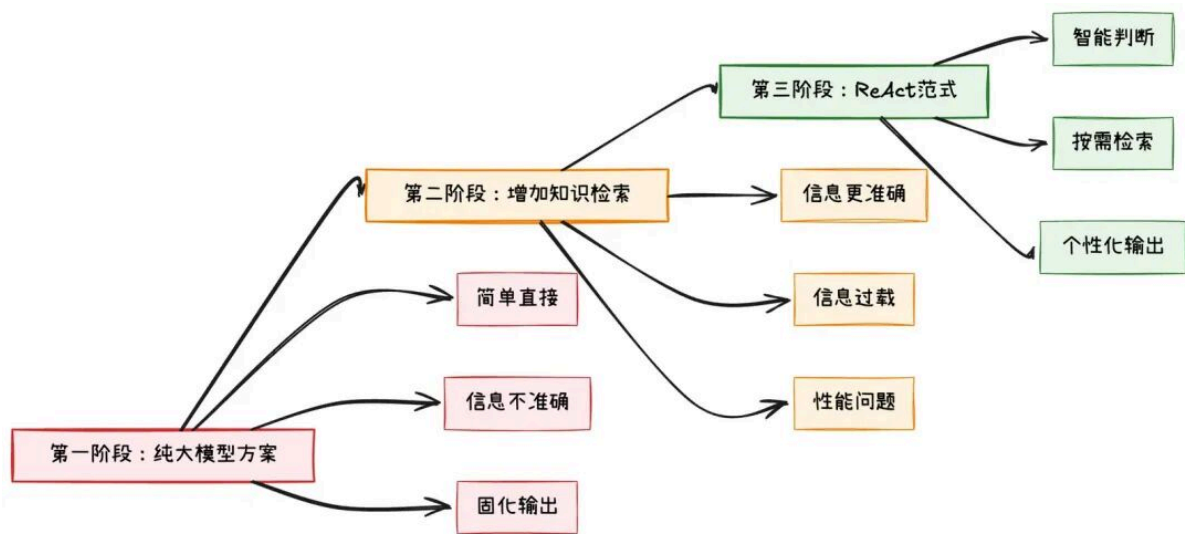
关键点：ReAct让AI能够像人一样，在不确定的情况下做出明智的决策。

- 分析用户需求（客户背景、产品需求、行业特点）
- 收集相关信息（行业趋势、竞品分析、产品特性）
- 生成结构化的PPT大纲
- 确保内容质量和业务价值

4.2 系统架构深度解析：基于真实需求的架构演进

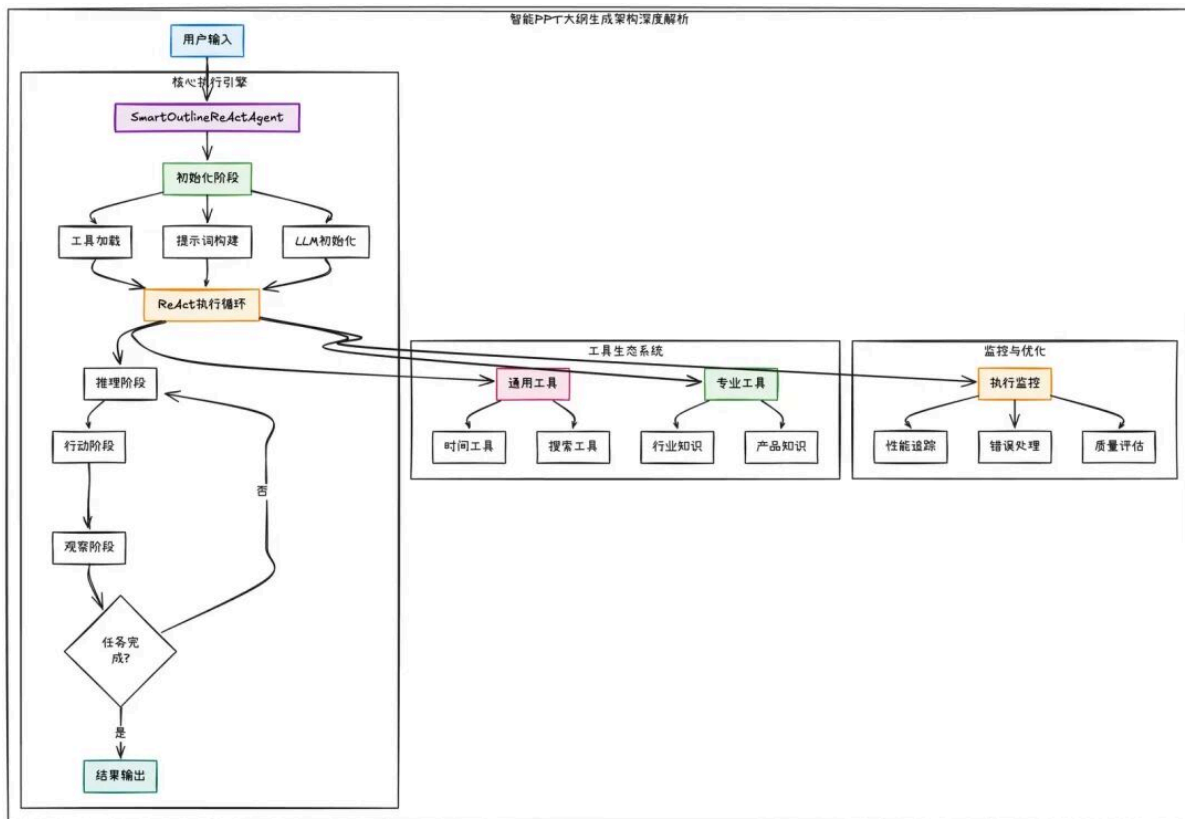
基于我在PPT生成项目中的真实落地经验，下面是 SmartOutlineReActAgent 的架构设计演进过程：

架构演进历程：



核心设计决策：

1. **智能判断层**：分析用户意图，决定是否需要检索
2. **按需检索层**：根据判断结果，选择性检索信息
3. **自适应生成层**：基于检索结果，生成个性化大纲



架构设计经验：

架构设计采用分层解耦设计，工具、提示词、执行引擎完全解耦，便于独立优化。整个执行过程由状态变化驱动，支持断点续传。通过通用工具 + 专业工具的分层设计，既保证通用性又满足专业性。同时需要从输入到输出的完整监控体系，确保系统稳定性。

```

class SmartOutlineReActAgent:
    """智能大纲生成ReAct Agent - 核心设计思路"""

    def __init__(self):
        # 核心组件：工具 + 模型 + Agent
  
```

```

self.tools = self._load_tools()          # 分层工具设计
self.llm = self._init_llm()              # 模型初始化
self.agent = self._create_react_agent()   # ReAct Agent

def _load_tools(self):
    """工具分层设计: 通用工具 + 专业工具"""
    general_tools = load_general_tools()   # 时间、搜索等
    internal_tools = load_internal_tools() # 内部知识库
    return general_tools + internal_tools

def _create_react_agent(self):
    """构建ReAct Agent - 核心执行引擎"""
    return create_react_agent(
        model=self.llm,
        tools=self.tools,
        prompt=self.system_prompt,
        version="v2" # 支持并行工具调用
    )

def generate_outline(self, user_input: str) -> str:
    """生成PPT大纲 - 核心业务逻辑"""
    # 1. 构建输入
    messages = [HumanMessage(content=user_input)]

    # 2. 执行ReAct循环
    result = self.agent.invoke({
        "messages": messages,
        "recursion_limit": 50,
        "max_iterations": 20
    })

    # 3. 返回结果
    return result["messages"][-1].content

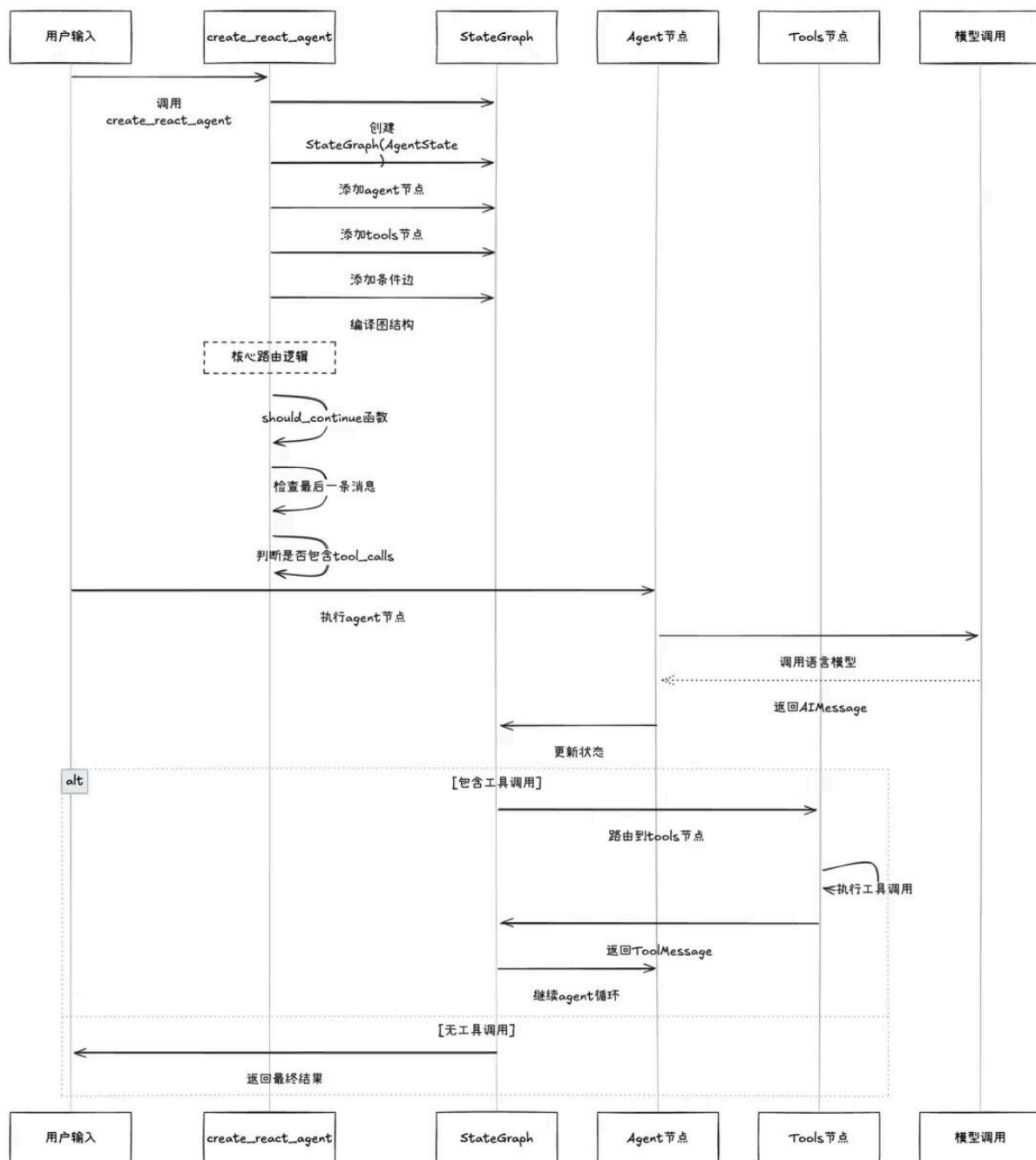
```

核心设计思路：

1. **分层架构**：工具、模型、Agent各司其职
2. **工具生态**：通用工具 + 专业工具的分层设计
3. **执行引擎**：ReAct循环 + 智能路由
4. **业务逻辑**：输入 → 推理 → 行动 → 输出

4.3 create_react_agent运行原则浅析

接下来先简单了解一下 `create_react_agent` 的内部实现，以便理解其内在的设计思路：



核心实现分析：

```
def create_react_agent(model, tools, prompt=None, version="v2"):
    """创建ReAct Agent的核心实现"""

    # 1. 状态模式定义
    class AgentState(TypedDict):
        messages: Annotated[Sequence[BaseMessage], add_messages]
        remaining_steps: int

    # 2. 图结构构建
    workflow = StateGraph(AgentState)

    # 3. 节点定义
    def agent_node(state: AgentState):
        """Agent节点 - 核心推理引擎"""
        # 绑定工具到模型
        model_with_tools = model.bind_tools(tools)
```

```

# 调用模型
response = model_with_tools.invoke(state["messages"])

return {"messages": [response]}

def tools_node(state: AgentState):
    """Tools节点 - 工具执行引擎"""
    if version == "v1":
        # v1版本: 串行执行所有工具
        return tool_node.invoke(state)
    else:
        # v2版本: 并行执行工具 (使用Send API)
        return tool_node.invoke(state)

# 4. 条件路由逻辑
def should_continue(state: AgentState) -> str:
    """智能路由决策"""
    messages = state["messages"]
    last_message = messages[-1]

    # 检查是否包含工具调用
    if isinstance(last_message, AIMessage) and last_message.tool_calls:
        return "tools"
    return "end"

# 5. 图结构组装
workflow.add_node("agent", agent_node)
workflow.add_node("tools", tools_node)
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {"tools": "tools", "end": END}
)
workflow.add_edge("tools", "agent")

# 6. 编译并返回
return workflow.compile()

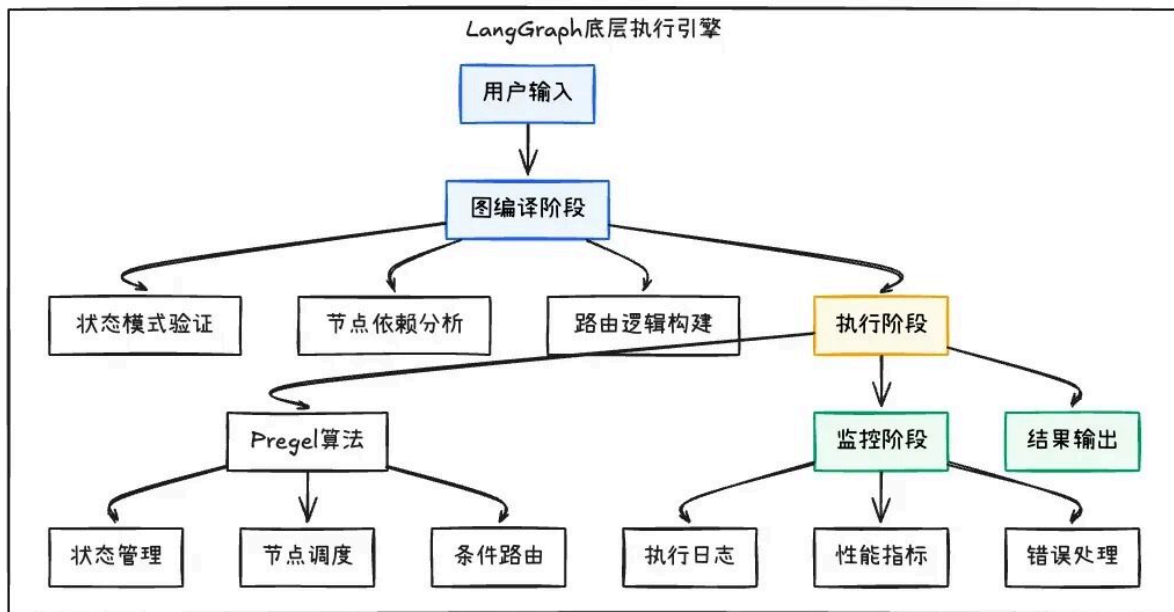
```

LangGraph不是简单的工具链，而是一个能够“思考”的智能系统。它通过状态管理让Agent记住整个对话过程，通过条件路由让Agent能够根据当前情况智能选择下一步，通过循环控制让Agent能够反思和优化结果。

让Agent从“执行者”变成了“思考者”，能够处理复杂的多轮对话和动态决策场景。

4.4 LangGraph底层执行引擎深度分析

继续深入分析LangGraph的底层执行机制，理解其如何实现高效的ReAct执行：



核心执行机制分析：

LangGraph底层执行引擎核心代码分析

class Pregel:

"""LangGraph的核心执行引擎 - 基于Pregel算法"""

def __init__(self, nodes: Dict[str, Any], edges: List[Edge]):

self.nodes = nodes

self.edges = edges

self.state_schema = self._build_state_schema()

def invoke(self, inputs: Dict[str, Any]) -> Dict[str, Any]:

"""执行图的核心方法"""

1. 状态初始化

state = self._initialize_state(inputs)

2. 执行循环

while not self._is_complete(state):

3. 节点调度

next_nodes = self._get_next_nodes(state)

4. 并行执行节点

updates = self._execute_nodes_parallel(next_nodes, state)

5. 状态更新

state = self._update_state(state, updates)

6. 检查终止条件

if self._should_terminate(state):

break

return state

def _execute_nodes_parallel(self, nodes: List[str], state: Dict) -> Dict:

"""并行执行多个节点 - 性能优化的关键"""

import asyncio

async def execute_node(node_name: str):

node_func = self.nodes[node_name]

return await node_func(state)

并行执行所有节点

tasks = [execute_node(node) for node in nodes]

```

results = asyncio.gather(*tasks)
return self._merge_results(results)

def _get_next_nodes(self, state: Dict) -> List[str]:
    """智能节点调度 - 基于条件路由"""
    next_nodes = []
    for edge in self.edges:
        if edge.condition(state):
            next_nodes.append(edge.target)
    return next_nodes

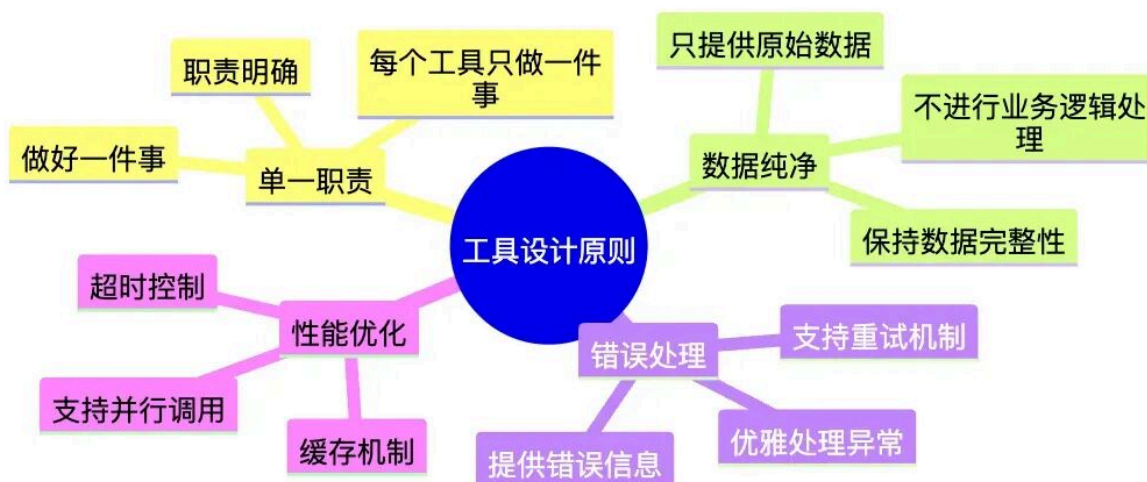
```

执行引擎的关键特性：

1. **图计算优化**：基于图计算算法，支持大规模并行计算
2. **状态管理高效**：增量状态更新，避免全量状态复制
3. **节点调度智能**：基于条件路由的智能节点调度
4. **并行执行支持**：支持多个节点的并行执行，提升性能
5. **错误恢复机制**：完整的错误处理和恢复机制

4.5 工具设计原则

基于实际项目经验，我总结出以下工具设计的核心原则：



实战案例：基于真实需求的工具设计

基于智能解决方案系统的真实需求，设计以下工具：

```

# 工具1：内部知识库搜索 - 解决信息获取问题
def search_internal_knowledge(query: str, knowledge_type: str = "all") -> str:
    """按需检索，避免信息过载"""
    # 根据知识类型选择性搜索
    if knowledge_type == "customer":
        results = customer_knowledge_base.search(query, limit=3)
    elif knowledge_type == "product":
        results = product_knowledge_base.search(query, limit=3)
    else:
        results = knowledge_base.search(query, limit=5)

```

```

# 返回原始数据，让AI自己判断如何使用
return format_search_results(results)

# 工具2: 用户意图分析 - 解决"固化输出"问题
def analyze_user_intent(user_input: str) -> str:
    """让AI理解用户的真实需求"""
    intent = {
        "needs_customer_info": "客户" in user_input,
        "needs_product_info": "产品" in user_input,
        "output_scope": "partial" if "只要" in user_input else "full",
        "page_count": extract_page_count(user_input)
    }
    return json.dumps(intent)

```

工具设计经验：

工具设计需要考虑职责边界、数据流纯净性、错误处理、性能优化和可观测性。每个工具都有明确的输入输出和职责范围，只负责数据获取和格式化，不进行业务逻辑处理。同时需要完善的异常处理、频率控制、并行调用等性能优化，以及完整的日志记录和监控指标。

```

# 工具设计核心原则
"""
1. 工具只负责获取信息，不负责分析
2. 工具只提供数据，不提供结论
3. 工具支持LLM推理，不替代LLM推理
4. 保持工具简单、纯净、单一职责
5. 工具的描述一定要详细
"""

@tool
def web_search(query: str) -> str:
    """获取互联网最新信息"""
    return search_engine.search(query)

@tool
def search_internal_knowledge(query: str) -> str:
    """获取内部文档和知识"""
    return knowledge_base.search(query)

```

工具设计原则：

工具设计要遵循单一职责原则，每个工具只做一件事，做好一件事。工具只提供原始数据，不进行业务逻辑处理，同时需要优雅地处理各种异常情况，支持并行调用和缓存机制。

4.6 提示词工程设计

在实际项目中，提示词工程是ReAct系统成功的关键因素。经过大量踩坑和调试，我总结出了一套提示词工程方法论：

4.6.1 核心工作原则设计

问题：传统的提示词往往过于僵化，无法适应复杂的业务场景。

解决方案：设计启发式指导框架，让Agent能够自主思考和决策。

核心工作原则

思考引导

- **智能分析**：分析用户输入的信息丰富度和需求复杂度
- **深度推理**：基于收集的信息进行多维度推理和关联分析
- **自主决策**：根据分析结果，自主选择执行策略和工具调用方式
- **价值导向**：每页内容都要体现客户价值和业务成果

深度思考框架

多维度分析策略：

- **客户维度**：规模、发展阶段、业务模式、技术成熟度
- **产品维度**：核心能力、技术优势、应用场景、实施复杂度
- **行业维度**：发展趋势、竞争格局、转型路径、成功要素
- **竞争维度**：竞品分析、差异化优势、价值主张、市场定位

4.6.2 提示词工程经验

从规则到框架的转变

传统提示词： "如果遇到X情况，执行Y操作"

优化后提示词： "分析当前情况，基于以下框架自主决策..."

从静态到动态的转变

传统提示词： "按照以下步骤执行"

优化后提示词： "根据实际情况，灵活调整执行策略"

从检查清单到质量框架的转变

传统提示词： "确保包含以下要素：A、B、C"

优化后提示词： "评估内容质量，确保达到以下标准..."

4.6.3 实践中的关键经验

经验1：提示词不是指令，而是思考引导

错误的提示词设计

prompt = ""

如果用户询问天气，调用天气API

如果用户询问时间，调用时间API

如果用户询问新闻，调用新闻API

""

正确的提示词设计

prompt = ""

分析用户需求，判断需要什么信息，然后选择合适的工具。

考虑以下因素：

1. 用户意图的明确程度
2. 所需信息的类型和来源
3. 当前上下文的相关性
4. 用户可能的后续需求

.....

经验2：建立质量评估框架，而不是检查清单

```
# 错误的质量检查
quality_check = [
    "是否包含所有必要信息？",
    "是否格式正确？",
    "是否长度合适？"
]

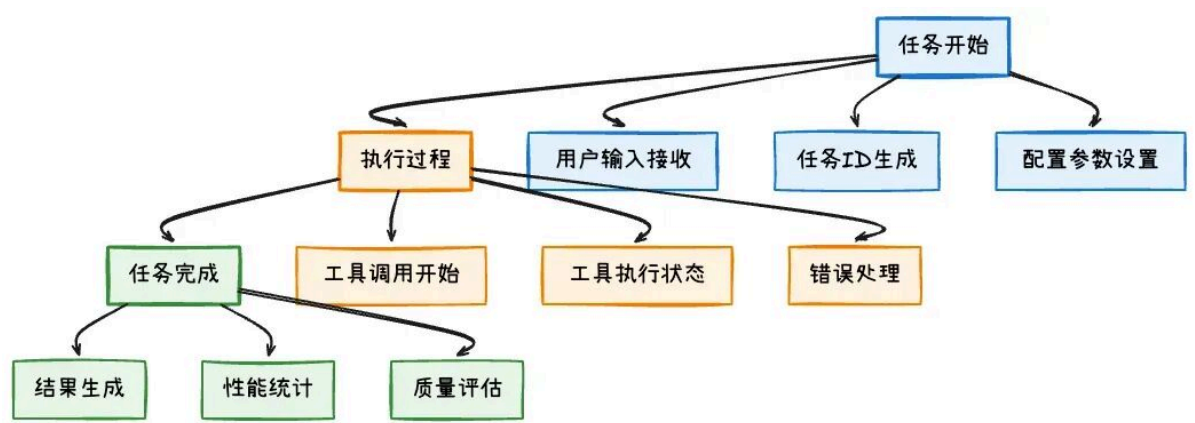
# 正确的质量框架
quality_framework = """
评估内容质量：
1. 信息完整性：是否回答了用户的核心问题？
2. 逻辑一致性：内容是否逻辑清晰、前后一致？
3. 价值相关性：是否对用户有实际价值？
4. 可操作性：用户是否能够基于此内容采取行动？
"""
```

经验3：场景适配比通用模板更重要

```
# 通用模板的问题
generic_prompt = "请分析以下内容并提供建议"

# 场景适配的优势
business_analysis_prompt = """
作为业务分析师，请分析以下内容：
1. 识别关键业务指标和趋势
2. 分析潜在风险和机会
3. 提供具体的改进建议
4. 考虑实施可行性和成本效益
"""
```

4.6.4 提示词优化的关键原则



启发式指导而非僵化规则

```
# 不好的提示词
prompt = """
你必须按照以下步骤：
1. 先分析问题
```

2. 再调用工具
 3. 最后给出答案
- ```
"""
```

# 好的提示词

```
prompt = """
```

你是一个智能助手，能够帮助用户解决各种问题。

当遇到需要外部信息的问题时，你可以主动调用相关工具。

请根据问题的具体情况，灵活地选择最合适的解决方案。

```
"""
```

## 灵活策略而非固定流程

# 不好的设计

```
def fixed_workflow(question):
 if "天气" in question:
 return call_weather_api()
 elif "订单" in question:
 return call_order_api()
 else:
 return "我无法处理"
```

# 好的设计

```
def flexible_workflow(question):
 # 让AI自己决定需要什么工具
 return agent.invoke({"messages": [{"role": "user", "content": question}]})
```

## 质量框架而非检查清单

# 不好的设计

```
def check_quality(response):
 checks = [
 "是否包含关键词",
 "是否超过100字",
 "是否包含标点符号"
]
 return all(checks)
```

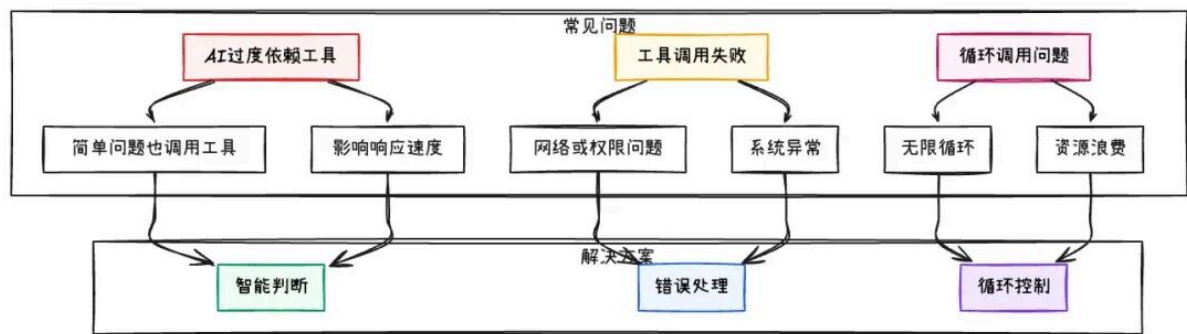
# 好的设计

```
def assess_quality(response, context):
 # 基于上下文和用户需求评估质量
 return quality_score
```

### 4.6.5 提示词优化辅助工具

- 谷歌提示库: [Generative AI prompt samples](#)
- 火山引擎提示词优化平台: [PromptPilot](#)

## 4.7 常见问题与解决方案



### AI过度依赖工具

```
解决方案：在提示词中强调判断能力
prompt = """
你是一个智能助手。当用户问题简单明确时，可以直接回答；
当需要外部信息时，才调用相关工具。
请根据问题的复杂度和信息需求，合理选择处理方式。
"""
```

### 工具调用失败处理

```
解决方案：优雅的错误处理
def robust_tool_call(tool_name, tool_input):
 try:
 result = tool_name.invoke(tool_input)
 return result
 except Exception as e:
 return f"工具调用失败: {str(e)}, 让我尝试其他方法"
```

### 循环调用问题

```
解决方案：设置最大迭代次数
class ReActAgent:
 def __init__(self, max_iterations=5):
 self.max_iterations = max_iterations
 self.current_iteration = 0

 def should_continue(self, state):
 self.current_iteration += 1
 if self.current_iteration >= self.max_iterations:
 return "end"
 # 其他逻辑...
```

## 4.8 性能优化实践

在智能解决方案系统中，我遇到了几个关键的性能问题，这些问题直接影响了用户体验：

### 信息过载导致的性能问题

```
问题：每次都要检索大量信息，即使不需要
```

```
def generate_outline_with_knowledge(user_input):
 # 问题：无论用户需要什么，都检索所有信息
 customer_info = search_customer_knowledge(user_input) # 耗时2-3秒
 product_info = search_product_knowledge(user_input) # 耗时2-3秒
 industry_info = search_industry_knowledge(user_input) # 耗时2-3秒

 # 总耗时：6-9秒，用户体验很差
 return generate_outline(customer_info, product_info, industry_info)

解决方案：ReAct智能判断
def react_outline_generation(user_input):
 # 1. 先分析用户意图
 intent = analyze_user_intent(user_input)

 # 2. 按需检索
 retrieved_info = {}
 if intent.needs_customer_info:
 retrieved_info["customer"] = search_customer_knowledge(user_input)
 if intent.needs_product_info:
 retrieved_info["product"] = search_product_knowledge(user_input)
 if intent.needs_industry_info:
 retrieved_info["industry"] = search_industry_knowledge(user_input)

 # 3. 基于检索结果生成大纲
 return generate_adaptive_outline(intent, retrieved_info)
```

## 固化输出导致的用户体验问题

```
问题：总是生成"完整"的PPT大纲
用户说："我只要3页介绍产品功能"
系统回答：生成20页的完整PPT大纲 # 用户体验差

解决方案：智能判断输出范围
def generate_adaptive_outline(intent, retrieved_info):
 if intent.output_scope == "partial":
 # 只生成用户需要的部分
 return generate_partial_outline(intent.focus_areas, retrieved_info)
 elif intent.output_scope == "full":
 # 生成完整大纲
 return generate_full_outline(retrieved_info)
 else:
 # 根据页数要求生成
 return generate_specific_outline(intent.page_count, retrieved_info)
```

## 工具调用频率过高

```
问题：AI过度依赖工具，即使问题很简单
用户问："什么是PPT？"
AI回答：我需要搜索一下PPT的定义... # 不必要的工具调用

解决方案：智能判断是否需要工具
def should_use_tools(user_input, intent):
 # 简单问题直接回答
```

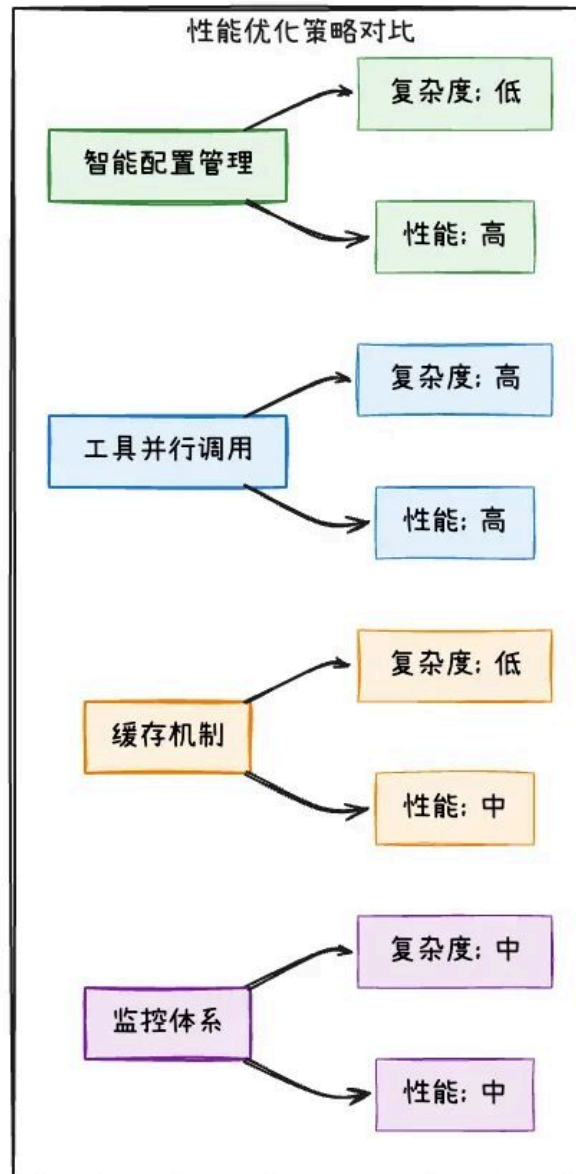
```

if intent.complexity == "simple":
 return False

需要外部信息才调用工具
if intent.needs_external_info:
 return True

return False

```



## 1. 智能配置管理

```

def get_adaptive_config(task_complexity: str) -> Dict[str, Any]:
 configs = {
 "simple": {
 "recursion_limit": 20,
 "max_iterations": 10,
 "timeout": 30
 },
 "complex": {
 "recursion_limit": 50,
 "max_iterations": 25,
 "timeout": 120
 }
 }

```

```

 }
}

return configs.get(task_complexity, configs["simple"])

```

## 2. 工具并行调用

```

支持并行工具调用
def parallel_tool_calls(tools: List[BaseTool], inputs: List[Dict]) -> List[str]:

 import asyncio

 async def call_tool(tool, input_data):
 return await tool.ainvoke(input_data)

 tasks = [call_tool(tool, input_data) for tool, input_data in zip(tools,
inputs)]
 return asyncio.run(asyncio.gather(*tasks))

```

## 3. 调用频率控制

```

import time
from collections import defaultdict

class RateLimiter:
 def __init__(self, max_calls_per_minute=60):
 self.max_calls = max_calls_per_minute
 self.calls = defaultdict(list)

 def can_call(self, tool_name: str) -> bool:
 now = time.time()
 # 清理1分钟前的调用记录
 self.calls[tool_name] = [t for t in self.calls[tool_name] if now - t <
60]

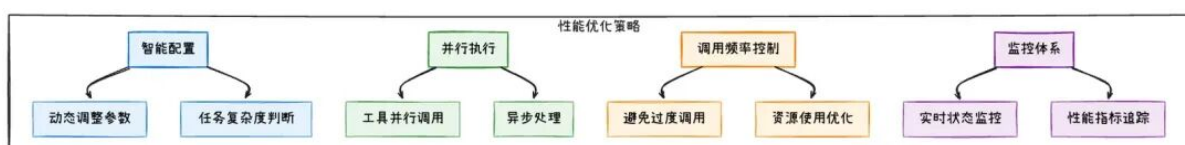
 return len(self.calls[tool_name]) < self.max_calls

 def record_call(self, tool_name: str):
 self.calls[tool_name].append(time.time())

创建全局频率限制器实例
rate_limiter = RateLimiter()

```

## 4.9 错误处理与监控



```

def _handle_state_update(self, data: Any, step_count: int, outline_id: str,
source: str = "", message_context: Dict[str, Any] = None):
 # 处理工具调用开始

```

```

 if isinstance(last_message, AIMessage) and last_message.tool_calls:
 for tool_call in last_message.tool_calls:
 send_outline_tool_call_start(
 outline_id=outline_id,
 tool_name=tool_name,
 tool_args=tool_args,
 step_count=step_count
)
 # 处理工具调用完成
 elif isinstance(last_message, ToolMessage):
 if is_error:
 send_outline_tool_call_error(...)
 else:
 send_outline_tool_call_complete(...)

```

**监控体系设计：**

1. **全链路监控**：从任务开始到完成的完整监控；
2. **实时日志**：详细的执行日志和状态更新；
3. **错误追踪**：完整的错误信息和堆栈跟踪；
4. **性能指标**：执行时间、工具调用次数、成功率等；

通过智能PPT生成系统的实践，我们积累了大量的优化经验，掌握了这些经验后，接下来我们回到基础，通过手写实现来深入理解ReAct的机制。

## 五、手写实践：从零构建ReAct Agent

经过前面的理论学习和源码分析，现在动手实现一个简化版的ReAct Agent。这个实现虽然功能简单，但包含了ReAct范式的要素。

### 5.1 核心实现代码

```

手写ReAct Agent
安装: pip install langchain langchain-ollama

from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage
from typing import Dict, List, Any, Optional
import re
import time

class MiniReActAgent:
 """手写ReAct Agent"""
 def __init__(self):
 self.model = ChatOllama(model="qwen3:8b", temperature=0.1)
 self.tools = {
 "time": self._get_time,
 "calc": self._calculate,
 "search": self._search
 }

状态管理

```

```

self.state = {
 "messages": [],
 "current_step": "start",
 "tool_results": {},
 "reasoning": ""
}

def _get_time(self, param=""):
 """获取当前时间"""
 import datetime
 now = datetime.datetime.now()
 return f"现在是{now.strftime('%Y年%m月%d日 %H:%M')}"

def _calculate(self, expression: str):
 """安全计算表达式"""
 try:
 if not re.match(r'^[0-9+\-*/.()]+$', expression):
 return "计算错误: 表达式包含非法字符"
 return str(eval(expression))
 except:
 return "计算错误: 表达式无效"

def _search(self, query: str):
 """模拟搜索功能"""
 return f"关于'{query}'的搜索结果..."

def should_continue(self, state: Dict) -> str:
 """条件路由 """
 last_message = state["messages"][-1] if state["messages"] else None

 # 检查AI回复中是否包含工具调用
 if isinstance(last_message, AIMessage):
 content = last_message.content
 if "工具:" in content and "参数:" in content:
 return "tools" # 需要工具
 return "end" # 直接结束

def agent_node(self, state: Dict) -> Dict:
 """Agent节点 """
 # 构建工具描述 - 让AI知道有哪些工具可用
 tool_descriptions = []
 for tool_name, tool_func in self.tools.items():
 if tool_name == "time":
 tool_descriptions.append("time: 获取当前时间(无需参数)")
 elif tool_name == "calc":
 tool_descriptions.append("calc: 计算数学表达式(需要表达式参数)")

 elif tool_name == "search":
 tool_descriptions.append("search: 搜索信息(需要搜索关键词)")

 # 构建提示词
 last_message = state["messages"][-1].content

 # 检查是否有工具执行结果

```

```

if "工具执行结果:" in last_message:
 # 基于工具结果继续推理
 prompt = f"""
 用户问题: {state['messages'][0].content}
 工具执行结果: {last_message}

 请基于工具执行结果回答用户问题。
 如果还需要更多信息, 请回答: 工具: [工具名] 参数: [参数]
 如果信息足够, 请直接回答用户问题。
 """

else:
 # 初始推理
 prompt = f"""
 用户问题: {last_message}
 可用工具: {'', '.join(tool_descriptions)}

 请分析问题并决定是否需要使用工具。
 如果需要工具, 请回答: 工具: [工具名] 参数: [参数]
 注意: time工具无需参数, 直接写"工具: time 参数: 无"
 如果不需要工具, 请直接回答用户问题。
 """

调用模型
response = self.model.invoke([HumanMessage(content=prompt)])

更新状态
new_state = state.copy()
new_state["messages"] = state["messages"] +
[AIMessage(content=response.content)]
new_state["current_step"] = "agent"

return new_state

def tools_node(self, state: Dict) -> Dict:
 """Tools节点 """
 last_message = state["messages"][-1]

 if isinstance(last_message, AIMessage):
 content = last_message.content

 # 解析工具调用
 if "工具:" in content and "参数:" in content:
 # 提取工具名和参数
 tool_match = re.search(r'工具:\s*(\w+)', content)
 param_match = re.search(r'参数:\s*(.+)', content)

 if tool_match and param_match:
 tool_name = tool_match.group(1)
 param = param_match.group(1).strip()

 # 执行工具
 if tool_name in self.tools:
 # 处理time工具的无参数调用

```

```

 if tool_name == "time" and (param == "无" or param == ""
or param == "[]"):
 result = self.tools[tool_name]()
 else:
 result = self.tools[tool_name](param)

 # 更新状态
 new_state = state.copy()
 new_state["messages"] = state["messages"] +
[AIMessage(content=f"工具执行结果: {result}")]
 new_state["current_step"] = "tools"

 return new_state

 return state

def react_cycle(self, question: str):
 """ReAct核心循环 """
 print(f"用户问题: {question}")

 # 初始化状态
 self.state = {
 "messages": [HumanMessage(content=question)],
 "current_step": "start",
 "tool_results": {},
 "reasoning": ""
 }

 # 执行循环
 max_iterations = 5
 for i in range(max_iterations):
 print(f"\n--- 第{i+1}轮执行 ---")

 # 1. Agent节点
 self.state = self.agent_node(self.state)
 print(f"Agent推理: {self.state['messages'][-1].content}")

 # 2. 条件路由
 next_step = self.should_continue(self.state)
 print(f"路由决策: {next_step}")

 if next_step == "tools":
 # 3. Tools节点
 self.state = self.tools_node(self.state)
 print(f"工具执行结果: {self.state['messages'][-1].content}")

 # 4. 继续下一轮推理
 continue
 else:
 # 5. 结束
 print("执行完成")
 break

```

```
return self.state["messages"][-1].content
```

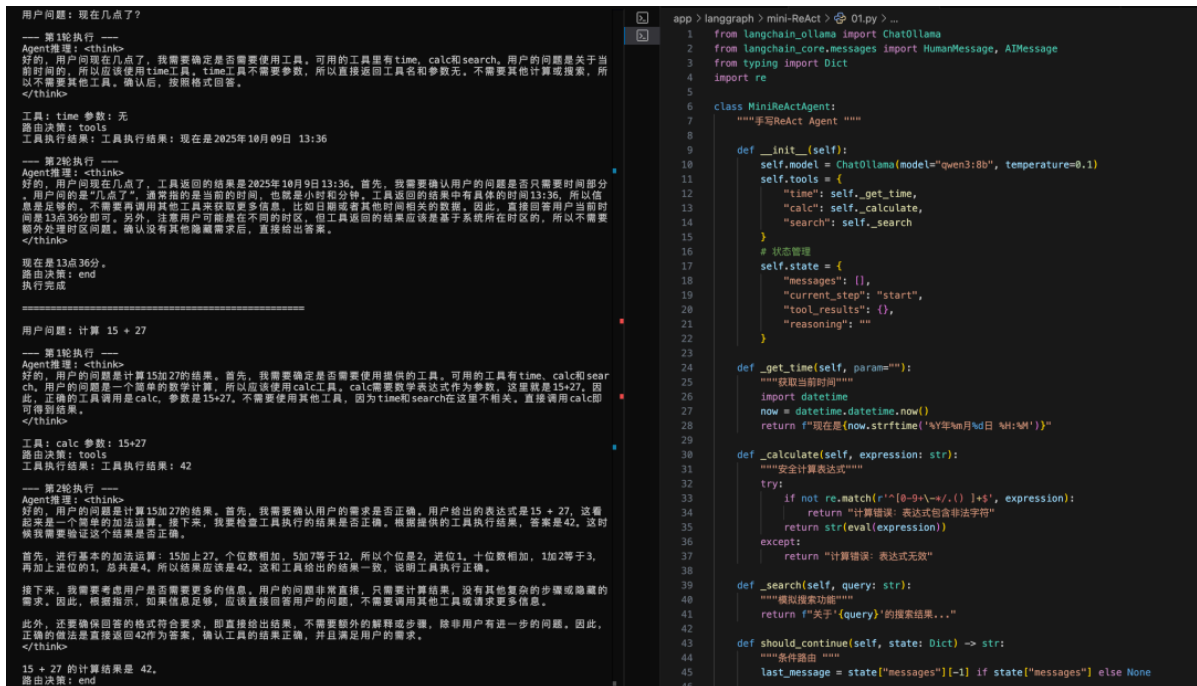
```
使用示例
if __name__ == "__main__":
 agent = MiniReActAgent()

 # 测试不同场景
 agent.react_cycle("现在几点了? ")
 print("\n" + "="*50 + "\n")

 agent.react_cycle("计算 15 + 27")
 print("\n" + "="*50 + "\n")

 agent.react_cycle("什么是人工智能? ")
```

运行效果展示：



从运行截图可以看到，这个极简ReAct Agent展现了ReAct范式的工作原理：当用户问“现在几点了？”时，Agent会推理出需要调用时间工具；当用户问“什么是人工智能？”时，Agent会直接回答而不调用工具。整个示例代码实现了推理→行动→观察→回答的完整循环。

## 5.2 从手写实现到生产系统的思考

手写实现确实能够帮助我们对ReAct有了更深入的理解。但说实话，真正要在项目中使用，这个简化版本还远远不够。

ReAct的“推理-行动-观察”循环看起来简单，但实现起来需要考虑很多细节。状态管理、工具调用、错误处理，每个环节都有坑。

在实际项目中，你需要考虑状态类型安全、工具调用失败、模型调用超时、内存管理等问题。这些细节处理不好，系统就会不稳定。

这就是为什么需要LangGraph这样的框架。它把这些复杂性都处理好了，让我们能够专注于业务逻辑。

## 六、总结

---

ReAct范式确实在Agent落地方面解决了很多实际问题，总结下来主要是这个方面：

### 可控性比智能性更重要

用户说"只要3页"，AI如果生成20页，即使内容再好，用户体验也是失败的。问题的根源在于AI的决策过程不透明，用户无法知道AI为什么生成了20页。ReAct让AI的思考过程变得透明，用户可以看到AI的推理过程，从而控制输出。这比让它变得更聪明更有价值。

### 工具设计决定系统上限

工具不是简单的API调用，而是AI的"手"和"眼"。设计工具时需要考虑灵活性、信息完整性和错误恢复能力。

### 状态管理是复杂系统的核心

LangGraph的图状态模型证明，复杂系统不是靠复杂的逻辑，而是靠清晰的状态转换。

### 从小做起

先实现一个最简单的ReAct Agent，让它能处理一个具体场景，再逐步增加复杂度。不要一开始就想着做复杂系统。

### 提示词需要反复调试

明确AI的角色和职责，提供清晰的决策标准，包含错误处理机制。这部分需要大量测试和调整。

### 性能优化从架构开始

在智能解决方案系统中，按需检索比全量检索快3倍，这就是架构设计的价值。这个例子说明，性能问题往往不是代码实现问题，而是架构设计问题。类似的还有缓存策略、并行处理、数据分片等，都需要从架构层面考虑。

### 技术选型的平衡

ReAct范式虽然强大，但并不是万能的。在实际项目中，需要根据任务复杂度选择合适的方案：**简单任务用直接API调用，复杂但确定的任务用workflow编排，只有真正需要动态推理和工具调用的场景才使用ReAct**。技术选型要务实，避免为了使用新技术而过度设计。

有些问题用ReAct解决得很好，有些问题用传统方法更简单。重要的是理解其原理，然后根据实际情况选择合适的技术方案。